

## Taller 2

Link al repositorio: <https://github.com/SergioAle210/ML-Engineering/tree/main/Talleres/Taller2>

### Captura de entorno creado con Python

```
^ ArchJapo // Japo // ~/ML-Engineering/Talleres/Taller2 // * [main] // v3.14.7
-> source .venv/bin/activate && python check_env.py && deactivate
python: 3.14.7
ejecutable: /home/Japo/Documents/Workspaces/uv/MLOPS/ML-Engineering/Talleres/Taller2/.venv/bin/python
sys.prefix: /home/Japo/Documents/Workspaces/uv/MLOPS/ML-Engineering/Talleres/Taller2/.venv
tipo: venv (python -m venv)

numpy      2.5.3
pandas     3.0.6
scipy      1.18.1
scikit-learn 1.9.1
joblib     1.6.0
matplotlib 3.11.2
seaborn    0.13.2
```

### Captura de entorno creado con Conda

```
^ ArchJapo // Japo // ~/ML-Engineering/Talleres/Taller2 // * [main] // v3.14.7
-> conda env list && conda activate taller2-ml && python check_env.py

# conda environments:
#
# * -> active
# + -> frozen
base                * /home/Japo/miniforge3
taller2-ml          /home/Japo/miniforge3/envs/taller2-ml

python: 3.12.14
ejecutable: /home/Japo/miniforge3/envs/taller2-ml/bin/python
sys.prefix: /home/Japo/miniforge3/envs/taller2-ml
tipo: conda

numpy      2.5.3
pandas     3.0.6
scipy      1.18.1
scikit-learn 1.9.1
joblib     1.6.0
matplotlib 3.11.2
seaborn    0.13.2
(taller2-ml)
```

### Conclusiones

- 1. Reproducibilidad de experimentos:** Un modelo depende tanto del código como de las versiones exactas de numpy, scikit-learn, etc. Un cambio menor (por ejemplo, un default distinto en `OneHotEncoder` o en un solver) cambia métricas o rompe la carga de un modelo guardado con `joblib`/`pickle`. Fijar el ambiente (`requirements-lock.txt`, `uv.lock`, `poetry.lock`) permite volver a entrenar y obtener los mismos resultados meses después.
- 2. Un ambiente por proyecto o por etapa:** En un mismo equipo pueden convivir proyectos con TensorFlow y PyTorch, o con versiones de CUDA incompatibles. Incluso dentro de un proyecto conviene separar el ambiente de exploración (Jupyter, matplotlib, seaborn), el de entrenamiento y uno

mínimo de serving, que solo necesita scikit-learn y pandas y produce imágenes Docker más pequeñas y seguras.

**3. Paridad entre desarrollo, CI y producción:** El mismo archivo de requisitos se usa en la laptop, en GitHub Actions, en el `Dockerfile` y en Databricks. Esto elimina el clásico «en mi máquina sí funciona». El reto del punto 3 muestra una forma de mantener una sola fuente de verdad: el paquete lee sus dependencias del mismo `requirements.txt` que usa el equipo.

**4. Elegir la herramienta según el tipo de dependencias:** `venv` + `pip` basta para la mayoría de proyectos de scikit-learn. Conda gana cuando hacen falta librerías nativas (CUDA, cuDNN, MKL, GDAL) porque las instala como paquetes. uv es la opción más rápida y moderna para proyectos puramente Python, con lockfile y manejo de versiones de Python incluidos. Poetry sigue siendo sólido para publicar librerías.

**5. Seguridad y auditoría:** Un ambiente declarado en un archivo se puede auditar (`pip-audit`, Dependabot), actualizar de forma controlada y revisar en un PR. En cambio, instalar paquetes a mano en el Python global no deja rastro y mezcla dependencias de proyectos distintos.